



Лекція 22

OAuth and OpenID Connect. Backend for Frontend

HTTP: особливості захисту

Протокол HTTP(S) - **stateless**, кожен запит опрацьовується окремо.

В кожному запиті має бути інформація, згідно з якою сервер прийматиме рішення, чи “безпечно” його обробляти.

Ця інформація може бути

- В **URL**
- В **Headers** (в т.ч. **Cookies**)
- В **body**
- На рівні встановлення **HTTPS** сесії (**mTLS**)

Методи захисту в web

HTTPS is a must

BASIC авторизація	Low
Login+password в URL або тілі запиту	Low
API-Key (header or param)	Medium
Сесії на рівні web-сервера (JSESSIONID і т.і.)	High
Bearer tokens	High
Mutual TLS (mTLS)	Higher

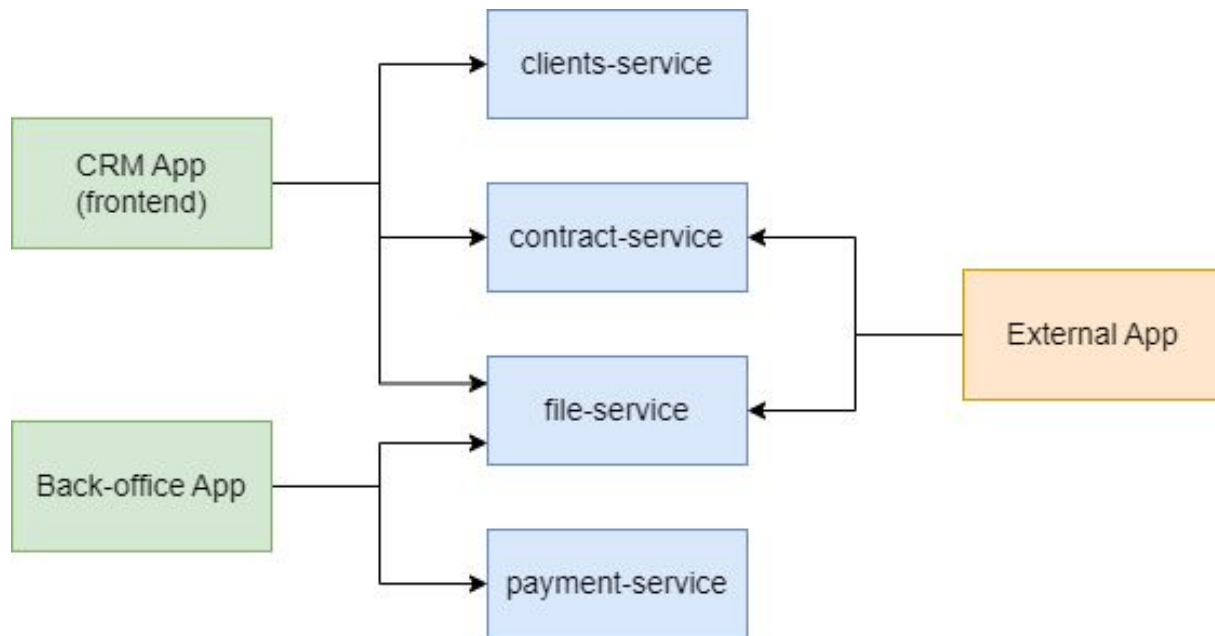
Bearer token

HTTP Header

Authorization: Bearer <token>

- Достатній для того, щоб сервер міг прийняти рішення, чи надавати доступ до ресурсу
- Часто має обмежений термін дії
 - від кількох хвилин до кількох годин
- Може бути пов'язаний з певним scope
 - Видається для доступу тільки до певного ресурсу

Як нам захистити нашу web-систему?



Де будуть дані користувачів? Що нам потрібно, щоб додати або відключити користувача?

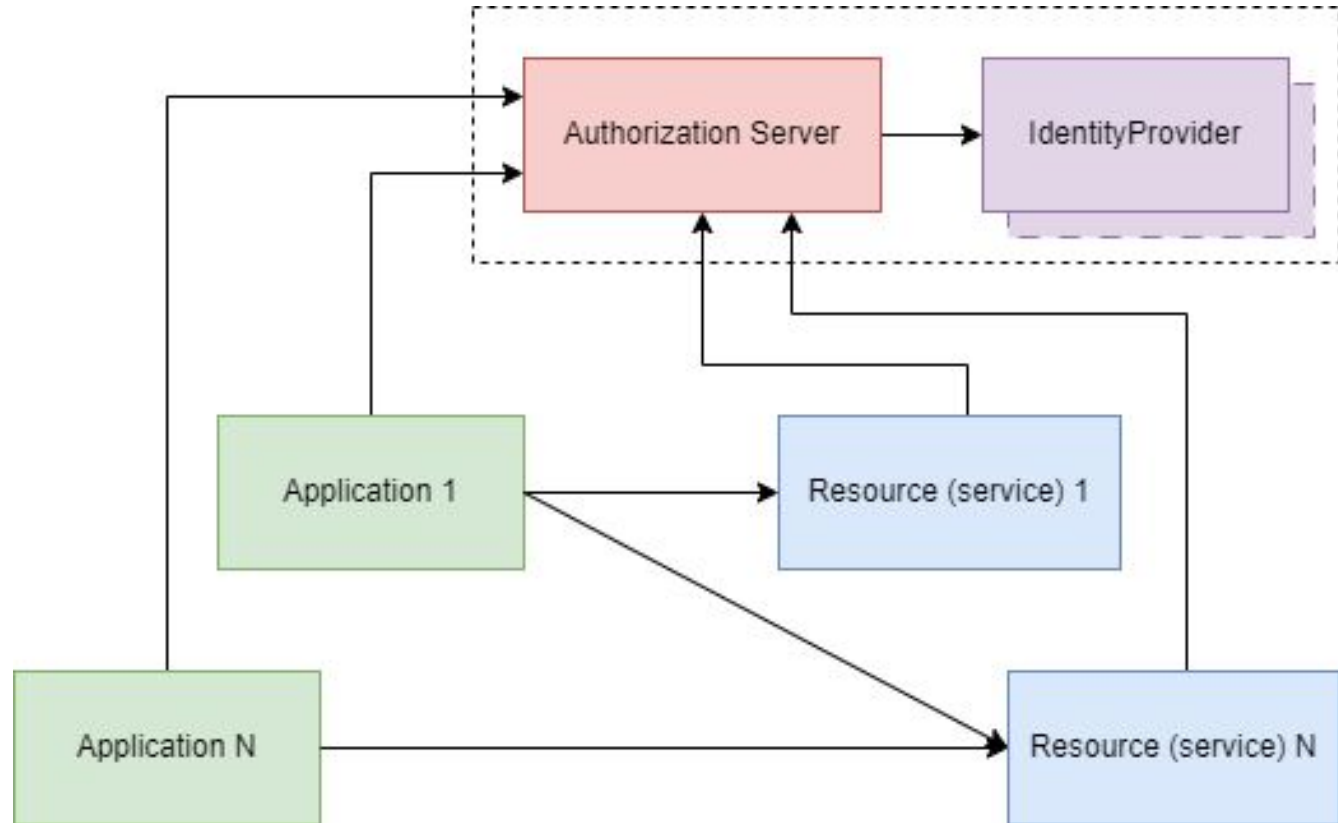
Де і як ми реалізовуємо логін? В кожному додатку? А що якщо ми захочемо додати MFA?

Чи можемо ми довіряти всім застосункам паролі користувачів?

Як керувати доступом до кожного з сервісів?

Як нам організувати Single Sign On (SSO)?

Authorization Server

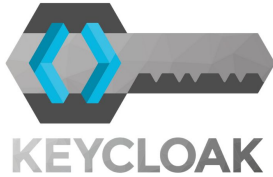


Authorization Server

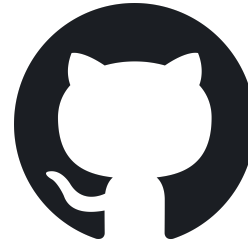
- “Знає” про всі Applications
 - Client Application
 - App Registration, etc.
- Або сам має інформацію про користувачів (імена, логіни, паролі), тобто є Identity Provider (IdP), або інтегрований з зовнішнім(и) IdP
- Видає і забезпечує валідацію токенів
- Може керувати сесіями (SSO, Logout, etc.)
- Може мати налаштування MFA
- Може підтримувати різні протоколи
 - OAuth 2.0, OpenID Connect, SAML

Existing solutions

Role-based authentication & identity



Basic authentication & identity



OAuth 2 and OpenID Connect

OAuth 2 (Open Authorization) - це **протокол авторизації**, який дозволяє користувачам надавати доступ до своїх ресурсів **стороннім застосункам** без потреби передавати свої логіни та паролі.

OAuth 2 стандартизує процес отримання і використання **токенів доступу** (access tokens), які використовуються для доступу до ресурсів.



OpenID Connect (OIDC) - це розширення OAuth 2.0, яке може надавати застосунку відомості про користувача, якого було автентифіковано, за рахунок передачі **ID-токенів**.

OAuth Roles



The User
Resource Owner



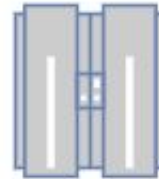
User Device
User Agent



The Application
Client



The API
Resource Server



Authorization server

Види передачі управління в web-застосунках

Back channel - HTTPS-запит від застосунку до серверу

- Надійний (перевірка сертифікату серверу)
- Неможливо перехопити або підмінити контент

Стосується як backend-backend, так і frontend-backend запитів (AJAX, Fetch)

Front channel - передача управління із використанням строки адреси браузера

```
window.location.href = 'http://www.someservice.com'  
response.sendRedirect("...");
```

Не такий надійний і безпечний, потребує додаткових механізмів захисту

Типи Клієнтських застосунків

Confidential Clients



Вміють зберігати
секрети

Public Clients



Не вміють зберігати
секрети

App Registration

App name: "My Application"

client_id: "myapp"

client_secret: "qz68P2T1aeQ73zvPoxs"

redirect_uri: "https://myapp.com"

(one o multiple)

Reference Tokens

Token	UserId	issue_time	expiration_time
Y6Ogbv1xKfM9mJRz3hyq	1	2023-03-21 10:00:00	2023-03-21 12:00:00
D5oEaWnRIBhQJ1vK0ZgH	1	2023-03-21 10:05:00	2023-03-21 12:05:00

Structured (self-contained) Tokens

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "sub": "1234567890",  "name": "John Doe",  "iat": 1516239022}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  your-256-bit-secret)
```

☐ secret base64 encoded

<https://jwt.io>

Перевірка GWT Access Tokenів

Resource server має “посилання” на Authorization Server і може завантажити (один раз) публічний ключ, який є парою приватного ключа, яким був підписаний токен.

При отриманні токена він

- розшифровує Signature частину цим ключем
- формує хеш від інформаційної частини
- порівнює ці значення
- перевіряє атрибути інформаційної частини
 - expiration time, etc.

OAuth Grant Types

Authorization Code + PKCE

Client Credentials

Device Code

Refresh Token

Implicit Flow (legacy)

Password Grant (legacy)

<https://oauth.net/2/grant-types/>

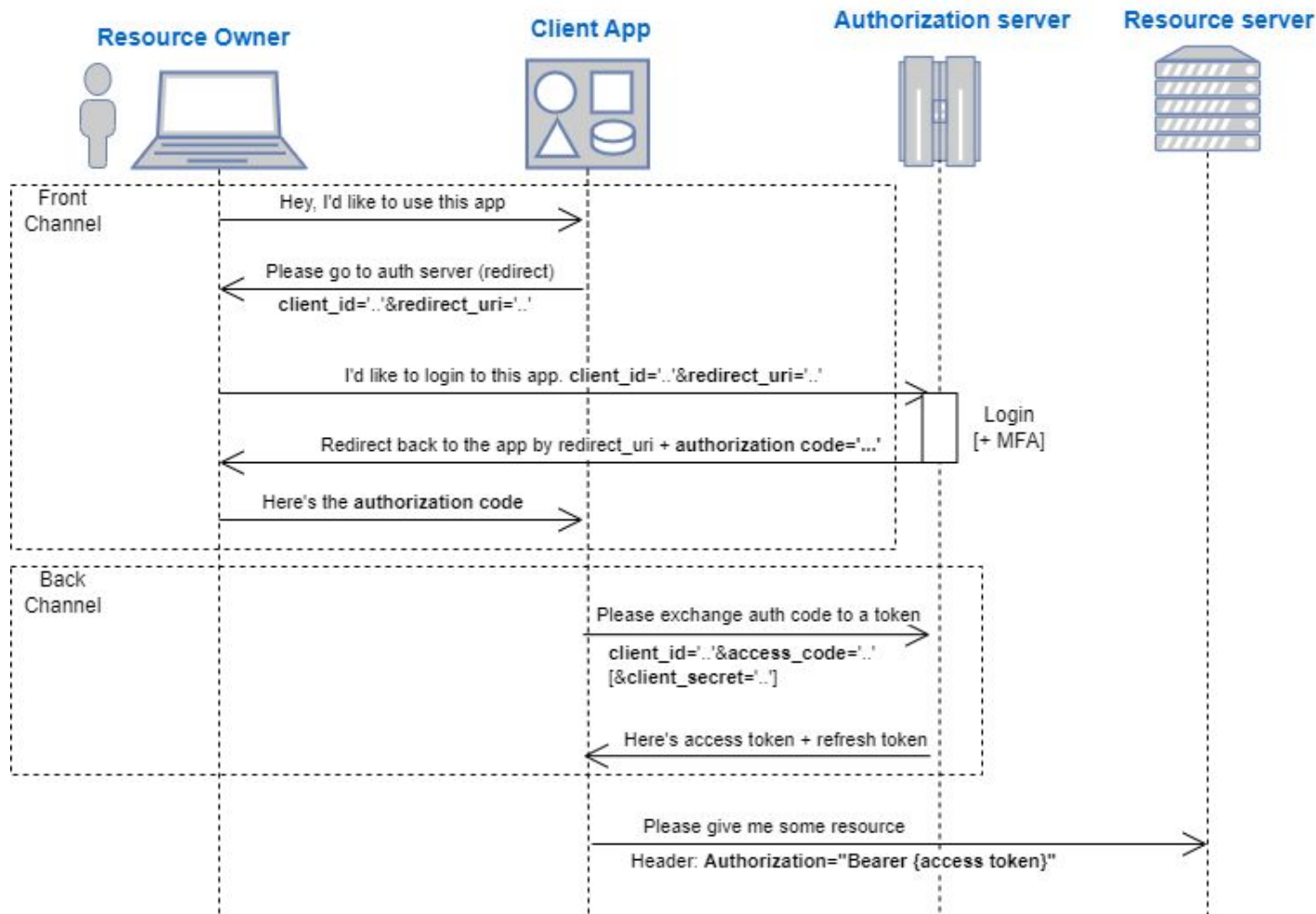
Password Grant flow

Client app приймає логін і пароль від користувача і обмінює їх на токени (access token, refresh) шляхом виводу відповідного endpoint

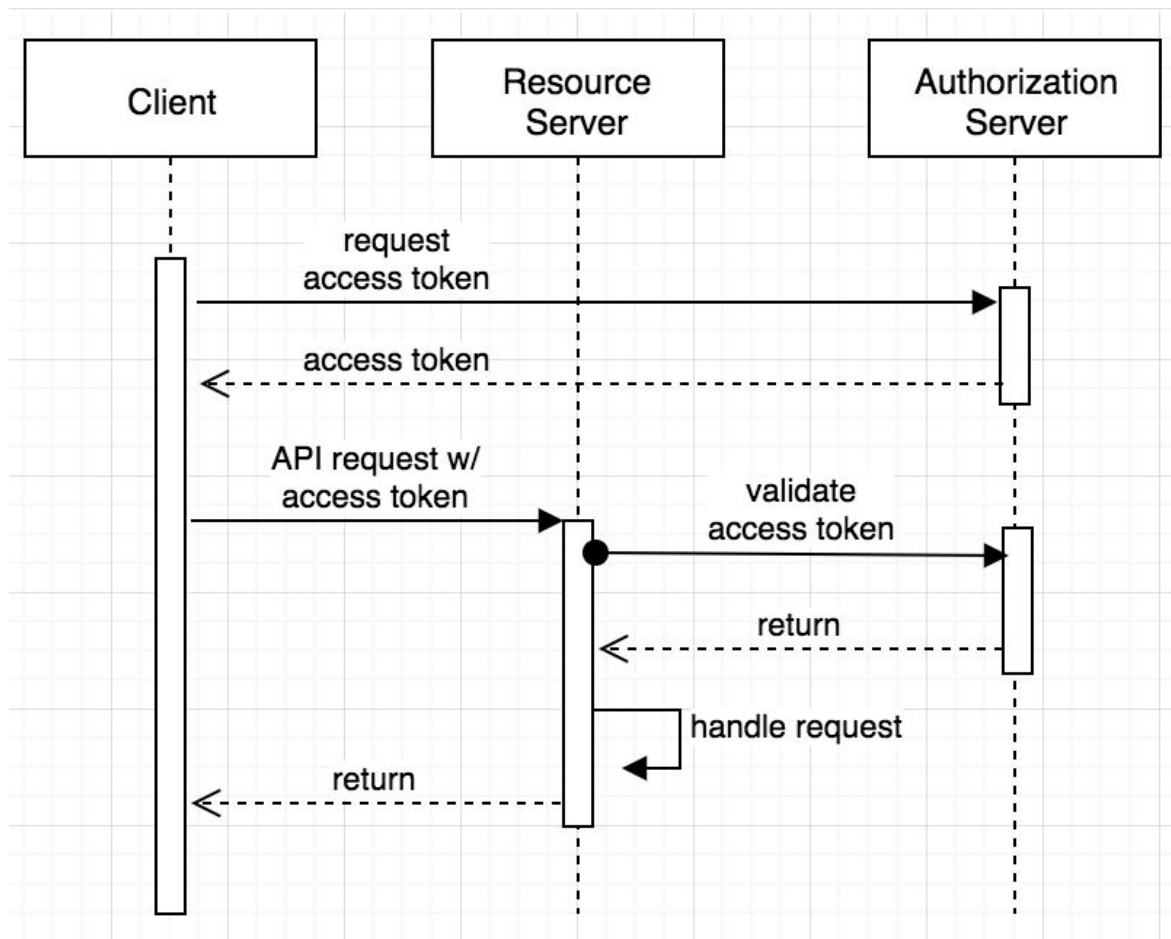
Що не так з Password Grant flow

1. Пароль проходить крізь Client App
 - Ризик витіку пароля (навмисного або ненавмисного, в т.ч. логування)
2. Неможливо побудувати SSO (Single Sign On)
3. Якщо нам буде потрібна MFA, її буде потрібно реалізовувати на стороні кожного Client App і значно складніше

Authorization Code flow (для людей)



Client Credentials Flow (для программ)



OpenID Connect (OIDC)

Client Application при авторизації та отриманні токенів вказує `scope: "openid"`

Authorization Server повертає **ID-токен** на додачу або замість Access-токену

- **ID-токен** - це JWT, що містить відомості про користувача (наприклад, email).

Застосування

- **Отримання відомостей про користувача:** Client Application використовує ці дані, щоб, наприклад, показати привітання
- **Автоматичний логін:** Якщо Client Application має власну, не обов'язково OAuth-based security, можна автоматично "логінити" користувача за його username (наприклад, email)

Види токенів

Access Token (Токен доступу)

- Використовується для доступу до захищених ресурсів від імені користувача.
- Має обмежений термін дії.
- Передається в запитах до API для підтвердження прав доступу.

Refresh Token (Токен оновлення)

- Використовується для отримання нового токена доступу після закінчення терміну його дії.
- Зазвичай має довший термін дії або навіть безстроковий.
- Передається в запитах до сервера авторизації для оновлення токена доступу.

ID Token (Ідентифікаційний токен)

- Використовується в рамках OpenID Connect (розширення OAuth 2.0).
- Містить інформацію про користувача та підтверджує його особу.
- Використовується для аутентифікації користувача.

Authorization Code (Авторизаційний код)

- Тимчасовий код, що використовується для отримання токена доступу.
- Генерується сервером авторизації після успішної аутентифікації користувача.
- Обмінюється на токен доступу та токен оновлення на сервері авторизації.

Consent Screen

Приложению
"Gateway Sample App"
нужны
дополнительные
права доступа к
вашему аккаунту
Google

 and.petrenko.a@gmail.com

 У приложения "Gateway Sample App" уже есть некоторые права доступа

Узнайте, к каким сервисам (Сервисов: 3) у приложения "Gateway Sample App" уже есть некоторые права доступа.

Убедитесь в надежности сервиса "Gateway Sample App"

Этот сайт или приложение сможет получить доступ к конфиденциальной информации. Ознакомьтесь с условиями использования и политикой конфиденциальности сервиса "Gateway Sample App", чтобы узнать, как будут обрабатываться ваши данные. Посмотреть или удалить приложения и сайты с доступом к вашему аккаунту можно на странице [Аккаунт Google](#).

[Подробнее об угрозах безопасности...](#)

Отмена

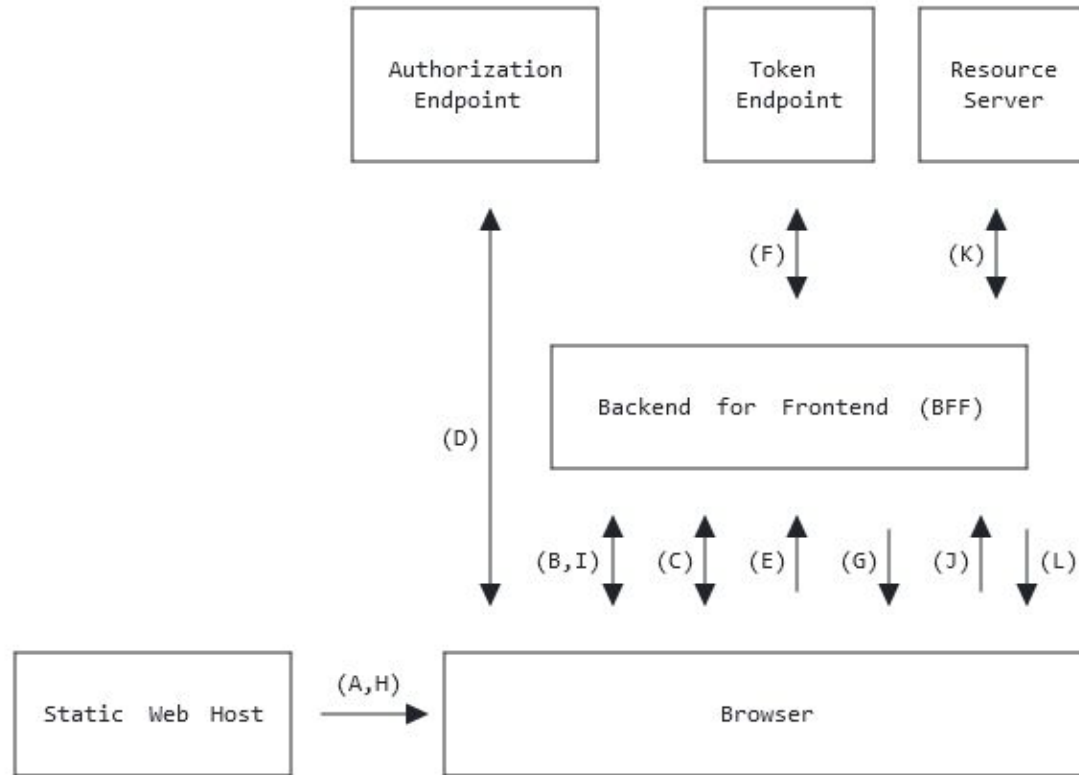
Продолжить

OAuth 2.0 for Browser-Based Applications

- **Backend-For-Frontend or BFF**
 - JavaScript application that relies on a backend component for handling OAuth responsibilities and proxies all requests through the backend component
- **Token-Mediating Backend**
 - JavaScript application that relies on a backend component for handling OAuth responsibilities, but calls resource servers directly using the access token
- **Browser-based OAuth Client**
 - A JavaScript application acting as the client, handling all OAuth responsibilities in the browser

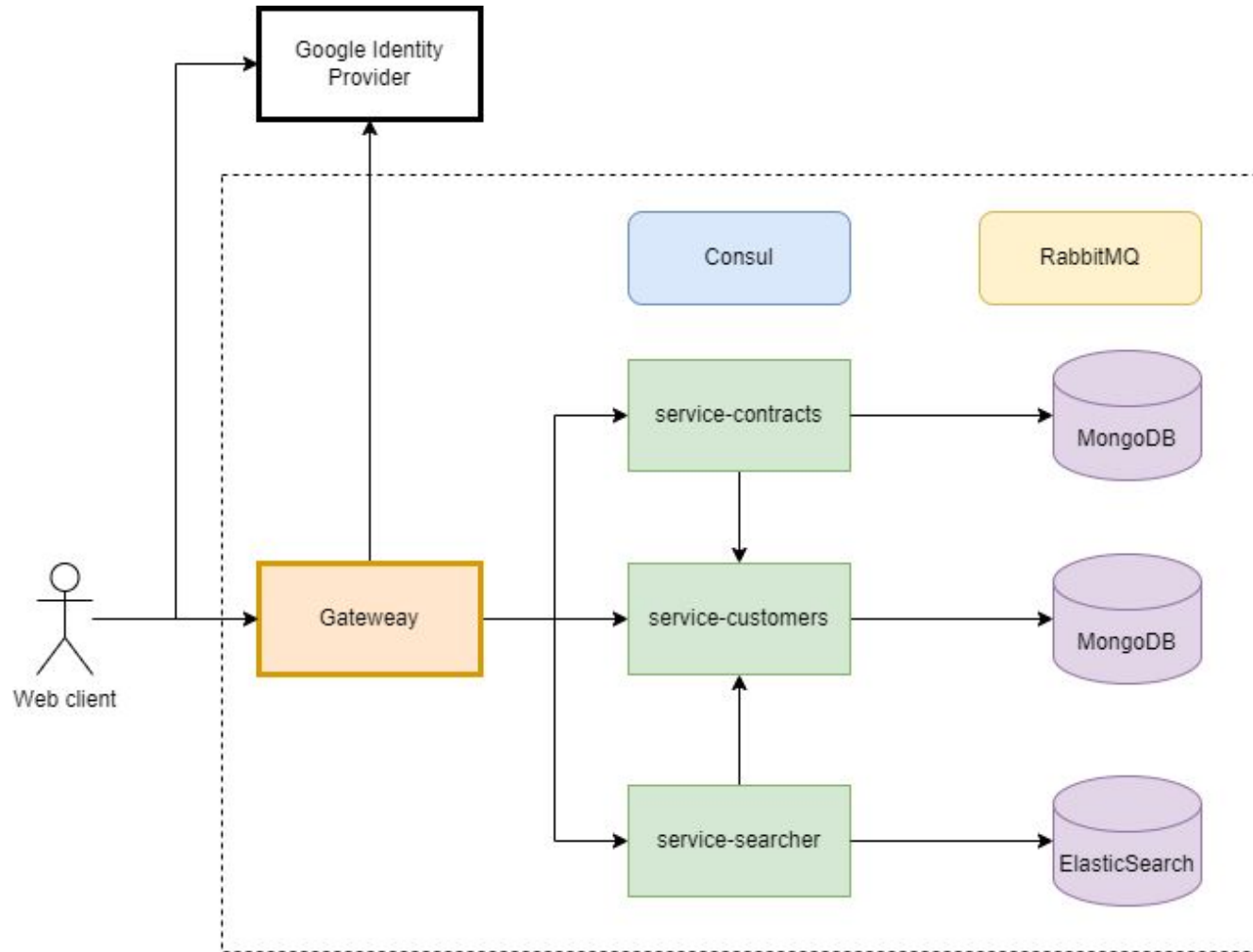
<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-browser-based-apps>

Backend-For-Frontend

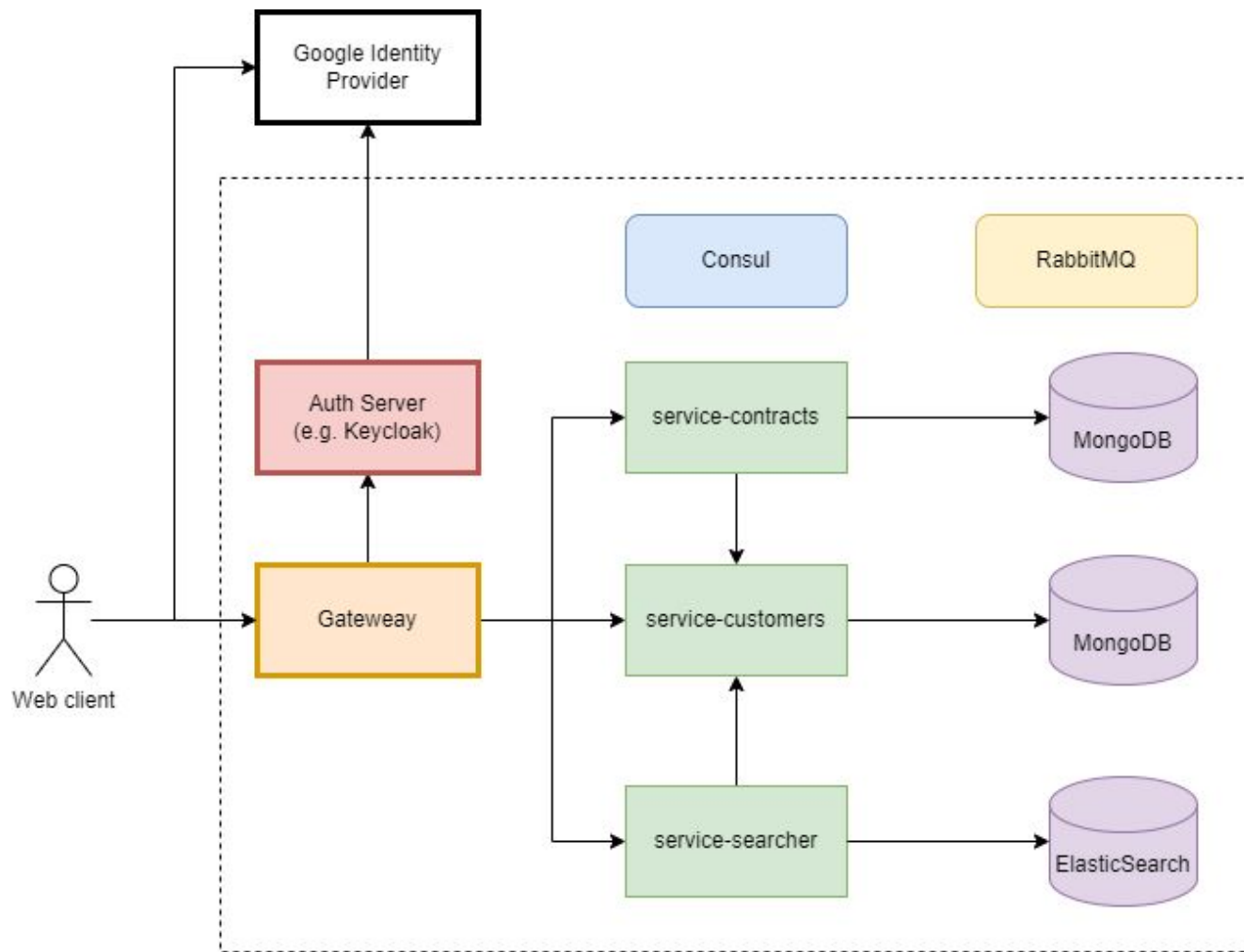


<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-browser-based-apps#name-backend-for-frontend-bff>

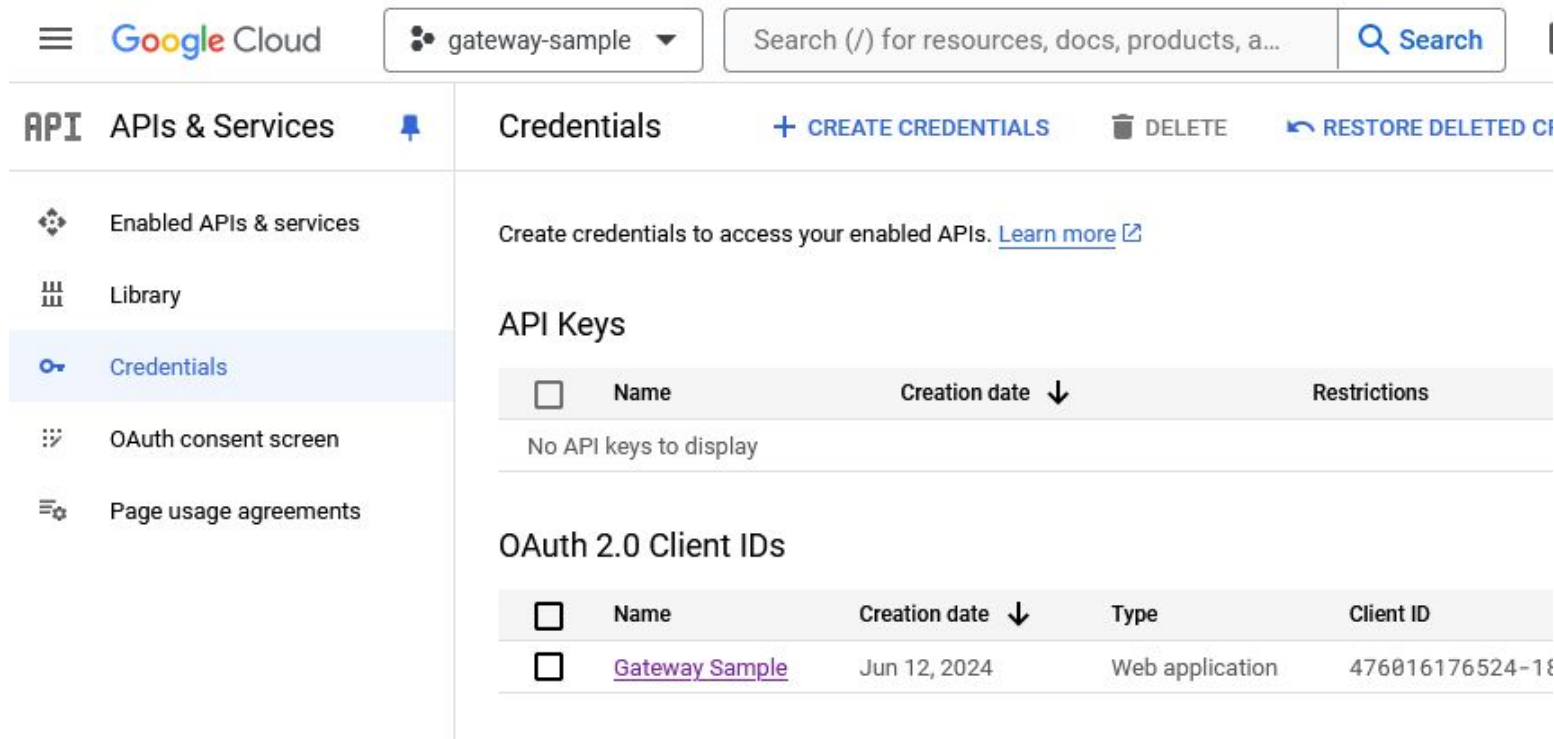
OIDC + BFF in Gateway Sample App



OAuth + BFF with better security control



Налаштування в Google Cloud Console



Google Cloud gateway-sample Search (/) for resources, docs, products, a... Search

API APIs & Services **Credentials** + CREATE CREDENTIALS DELETE RESTORE DELETED C

Enabled APIs & services
Library
Credentials
OAuth consent screen
Page usage agreements

Create credentials to access your enabled APIs. [Learn more](#)

API Keys

☐	Name	Creation date ↓	Restrictions
No API keys to display			

OAuth 2.0 Client IDs

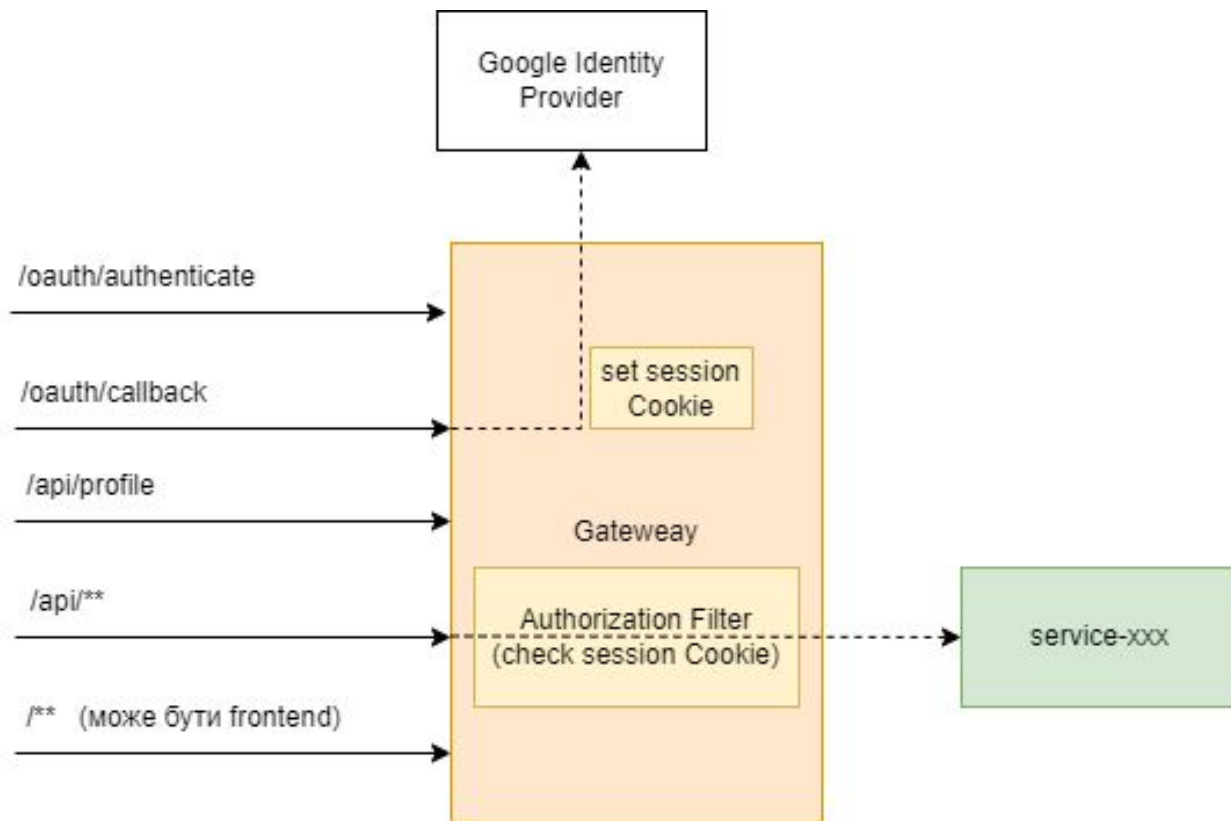
☐	Name	Creation date ↓	Type	Client ID
☐	Gateway Sample	Jun 12, 2024	Web application	476016176524-14

<https://developers.google.com/identity/openid-connect/openid-connect?hl=ru#java>

<https://keycloakthemes.com/blog/how-to-setup-sign-in-with-google-using-keycloak>

(див. розділ Create a Google Application)

FBV на практиці



Основні принципи BFF

1. Фронтенд завантажується і одразу робить запит на будь-який endpoint (наприклад, /profile), і отримує **401 Unauthorized**
2. Фронтенд показує віконце з кнопкою, що пропонує залогінитись
3. Кнопка веде на URL гейтвею, що ініціює логін (в прикладі /oauth/authenticate). Він генерує state і записує його у спец. Cookie, після чого робить **редірект на Authorization server з client_id, scope, redirect_uri, state.**
4. Після проходження автентифікації користувач редиректитується на вказаний redirect_uri (в прикладі /oauth/callback). Gateway перевіряє state, **обмінює Auth Code на ID Token**, парсить його, отримує дані користувача, генерує сесію і записує **SESSION-ID в Cookie (httpOnly=true)**. Робить редірект назад на frontend.
5. Frontend знов завантажується (так само як в п.1), робить запит на /profile, але за рахунок того що передається **SESSION-ID Cookie**, повертає нормальну відповідь. Frontend далі рендерить сторінку як для залогіненого користувача.